DISTRIBUTED HAZARD

# A batch job can poison your cache in every region at once.

How write-ahead log replication lag in Debezium and Kafka corrupts distributed Redis caches, breaks read-after-write consistency, and serves ghost data to shoppers.

**25+ Seconds**

Replication offset lag surge triggered when bulk primary updates flood Postgres WAL queues.

**Ghost Reads**

Outdated prices and sold-out inventory served by edge caches while the primary database holds new state.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh

SWIPE →

FAILURE STATE: SILENT INCONSISTENCY

# Primary commits in 2ms. Edge reads yesterday's state.

Asynchronous CDC decouples writes from cache synchronization. Under bulk write volume, consumer lag turns reads into ghost data.

1 **Batch Update Modifies 50,000 Rows**
An enterprise catalog sync executes a bulk UPDATE query on PostgreSQL. Transactions commit rapidly with fresh Log Sequence Numbers (LSN).

2 **Logical Replication Slot Balloons**
PostgreSQL generates 50MB/s of WAL records. The replication slot retains unconsumed WAL chunks, causing consumer lag to climb by 25 seconds.

3 **Debezium & Kafka Consumer Contention**
Event serialization buffers saturate. Edge read microservices bypass the database to query local Redis clusters for low latency.

4 **Edge Caches Serve Corrupted Ghost State**
Shoppers view obsolete discount prices or phantom inventory. No errors fire in API monitoring dashboards.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh

STRUCTURAL MODEL COLLISION

# Strictly Ordered WAL vs Clock-Time Cache Writes

PostgreSQL guarantees strict sequential mutation order via LSN. Redis applies writes on network arrival without sequence validation.

SOURCE ENGINE

### PostgreSQL WAL Stream

> **Ordering:** Strict linear sequence governed by 64-bit monotonic LSN.

> **Concurrency:** Serialized WAL records preserve exact causal order.

> **Durability:** Commits succeed only after WAL flush.

> **Invariant:** State changes strictly advance the LSN pointer.

DESTINATION CACHE

### Distributed Redis Cache

> **Ordering:** Unordered arrival time across network workers.

> **Concurrency:** Retried Kafka events arrive out of temporal sequence.

> **Durability:** Volatile in-memory keys without version checks.

> **Vulnerability:** Stale retries overwrite newer state without warning.

### The Fundamental Ordering Gap

A network hiccup or consumer partition rebalance causes event N+1 to process before retried event N. Redis blindly writes event N, permanently poisoning the key with obsolete state.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh

OPERATIONAL DRAG

# Lagging cache replicas cause direct revenue leakage.

Cache inconsistencies undermine commerce systems and risk infrastructure storage collapse.

### 01. Checkout Pricing Collisions

Shoppers add discounted products from stale cache records. At payment finalization, the primary database enforces updated pricing, leading to high abandonment and disputes.

### 02. Phantom Inventory Overselling

Stale inventory keys confirm stock availability for exhausted inventory, causing unfulfillable orders, negative customer sentiment, and operational penalty fees.

### 03. Primary Storage Exhaustion Risk

As Debezium lag grows, PostgreSQL replication slots retain gigabytes of unconfirmed WAL logs. Uncapped slots can fill primary disks, triggering emergency database read-only locks.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh

SWIPE →

PRODUCTION ARCHITECTURE

# Three gates to enforce read-after-write consistency.

Eliminate cache poisoning by binding cache hydration directly to database Log Sequence Numbers.

### 1 Monotonic LSN Verification

Store the database LSN with every cached key. Execute Redis writes via an atomic Lua script that compares the event LSN against the cached LSN. Reject any write where incoming LSN is less than or equal to current cache LSN.

### 2 Batch Compaction Filters

Collapse multiple row mutations in bulk update jobs into a single final state before pushing to Kafka. Drop no-op writes where the row hash is identical to reduce CDC replication traffic by 65%.

### 3 Asynchronous Poison Quarantines

Configure API gateways to inspect replica lag. When CDC lag exceeds 1,500ms, trip a circuit breaker to bypass cache and read directly from the primary database, while capping WAL retention via max_slot_wal_keep_size.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh

IMMEDIATE ACTION REQUIRED

# Inspect your replication slots today.

Execute this SQL command against your primary PostgreSQL cluster to identify replication lag and WAL disk accumulation:

```
/* Query active replication slots, consumer lag, and retained WAL */ SELECT slot_name, plugin, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal, pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS unconfirmed_bytes FROM pg_replication_slots; /* Atomic Redis Lua Guard Snippet: */ /* if redis.call('hget', KEYS[1], 'lsn') < ARGV[1] then */ /* redis.call('hset', KEYS[1], 'val', ARGV[2], 'lsn', ARGV[1]) */ /* end */
```

### Production Verification Checklist

> Verify Redis hydration scripts enforce atomic monotonic LSN checks.

> Set max_slot_wal_keep_size to prevent catastrophic disk exhaustion.

> Enforce read-after-write headers on transactional checkout endpoints.

**Ariya Sarrafzadeh**
Solutions Architect & Technical Product Lead
linkedin.com/in/ariya-sarrafzadeh